

Relay: Tour Presentation in Diff Views

Sep 23, 2026 · @Eric Staples

Summary

Tour highlights stop painting line backgrounds. The diff keeps its colors, and the tour presents through non-background channels: a gutter-edge rail, a focus outline, dimmed context text, an inline beat label, and an overview ruler mark. Beats that focus on removed code get a real, decoratable view of that code: the original side of a side-by-side diff when one is showing, otherwise a companion editor opened beside the diff.

Problem

Today the presenter decoration fills the whole anchor range with a background tint. Inside a diff that collides with added, removed, and modified line backgrounds:

- Added lines lose contrast under the tint.
- The anchor covers the whole function, so nothing shows which lines the narration sentence is about.
- In the inline diff, removed lines render in view zones that extensions can't decorate, so a beat about deleted code can't point at it at all.

Scope

- **In:** editor decorations for the presenter, the anchor model change they need, removed-code presentation, and state-dependent rendering.
- **Out:** panel layout, the agent protocol, and review state. Those are implemented per the [narration side panel spec](#) and don't change here.
- **Supersedes:** "Editor integration › Presenter pointer" in that spec. "Opening the right view" is extended rather than replaced.

Visual channels

The diff and the tour never write to the same visual property. The diff owns line and character backgrounds. The tour owns every other channel, so the two stack without blending.

Current. Tour tint fills every head line and fights the diff colors. No sign of which lines the beat is about.

```
scheduler.go (a41c9e) ↔ scheduler.go (7f02b1)

142 func (s *ControlPlaneNodeScheduler) listManagedControlPlanePending(ctx context.Context, cluster db.Cluster) ([]db.Node, error) {
143     pendingNodes, err := s.db.ListNodes(ctx, db.NodeFilters{
144         ClusterID:    &cluster.ID,
145         ControlPlane: ptr.To(true),
146         Pending:       ptr.To(true),
147         Deleting:      ptr.To(false),
148     })
149     if err != nil {
150         return nil, err
151     }
152
153     managed := make([]db.Node, 0, len(pendingNodes))
154     fmt.Println("Test adding a line to see how it shows up in the diff")
155     for _, node := range pendingNodes {
156         if !node.IsManagedByControlPlaneController() {
157             continue
158         }
159         // Migration only: keep pre-existing rows out of scheduling while their nil
160         // failure domains are being backfilled. Remove once every control-plane
161         // node is guaranteed to have a non-nil failure_domain.
162         if !node.HasFailureDomainDecision() {
163             continue
164         }
165         managed = append(managed, node)
166     }
167     return managed, nil
168 }
```

Proposed. Violet rail on context, focus box on the beat's lines, dimmed context text, beat label. Diff backgrounds untouched; removed lines (view zone) can't be decorated and stay as-is.

```
scheduler.go (a41c9e) ↔ scheduler.go (7f02b1)

142 func (s *ControlPlaneNodeScheduler) listManagedControlPlanePending(ctx context.Context, cluster db.Cluster) ([]db.Node, error) {
143     pendingNodes, err := s.db.ListNodes(ctx, db.NodeFilters{
144         ClusterID:    &cluster.ID,
145         ControlPlane: ptr.To(true),
146         Pending:       ptr.To(true),
147         Deleting:      ptr.To(false),
148     })
149     if err != nil {
150         return nil, err
151     }
152
153     managed := make([]db.Node, 0, len(pendingNodes))
154     fmt.Println("Test adding a line to see how it shows up in the diff")
155     for _, node := range pendingNodes {
156         if !node.IsManagedByControlPlaneController() {
157             continue
158         }
159         // Migration only: keep pre-existing rows out of scheduling while their nil
160         // failure domains are being backfilled. Remove once every control-plane
161         // node is guaranteed to have a non-nil failure_domain.
162         if !node.HasFailureDomainDecision() {
163             continue
164         }
165         managed = append(managed, node)
166     }
167     return managed, nil
168 }
```

Channel	Owner	Tour use
Line background	Diff	None
Character background	Diff	None
Left edge of content area	Tour	3px rail on every context line

Channel	Owner	Tour use
Line border box	Tour	1px outline around each focus span
Text opacity	Tour	Context lines outside the focus at 0.45
After-text attachment	Tour	Beat label on one focus line
Overview ruler, left lane	Tour	Rail color over the context range
Hover	Tour	Beat narration and claim links on focus lines

Color

The tour uses violet. Red and green belong to the diff, and blue is already taken by modified-line and selection semantics in most themes. Every color is a contributed theme color, so theme authors and high-contrast modes can override it.

Why opacity works here

Decoration `opacity` applies to the text span, not to the line's background layer. Dimmed lines keep full-strength diff backgrounds, so changed lines stay findable at a glance while the reviewer reads the focus.

Anchor model change

A beat anchor splits into a context range (the unit being discussed, usually a function) and zero or more focus spans (the lines the beat's sentence is about). Each focus span names its own side, so one beat can point at removed and added lines together.

```

interface Anchor {
  path: string;
  side: "base" | "head" | "worktree"; // side of the context range
  rev: string;
  context: Range;                      // was `range`
  focus?: FocusSpan[];
  symbol?: string;
  contentHash: string;                 // hash of context text at rev
}

interface FocusSpan {
  side: "base" | "head";
  range: Range;                       // in that side's line numbers
  kind?: "added" | "removed" | "unchanged"; // optional hint; computed from
hunks if absent
}

```

Compatibility

- Existing dossiers with `range` load as `context = range` and no focus. They render as rail-only beats, which is still an improvement over the background fill.
- `contentHash` keeps hashing the context text, so drift detection is unchanged.
- Each focus span is re-hashed separately on activation. If a span no longer matches, the beat still renders with the rail, and the panel shows the stale note from the existing drift flow.

Authoring guidance for the skill

- Keep focus spans tight: the lines the sentence makes a claim about, not the whole hunk.
- Omit focus when the beat genuinely describes the whole unit ("this function now filters pending nodes").
- A beat about a deletion puts `side: "base"` on its focus span, even if the context is on the head side.

Decoration set

The presenter uses ten decoration types (the nine below plus `railStale`, a dashed copy of rail), created once at activation and reused across editors. Everything below uses `createTextEditorDecorationType`, `setDecorations`, and `ThemeColor`, all stable API.

Type	Options	Applied to
<code>rail</code>	whole-line, left border 3px, overview ruler left lane	Every context line
<code>dim</code>	<code>opacity: 0.45</code>	Context lines outside every focus span
<code>boxTop</code> / <code>boxMid</code> / <code>boxBot</code> / <code>boxOne</code>	whole-line borders drawn so a multi-line span reads as one box	Focus span lines
<code>label</code>	<code>after</code> attachment with background, set per range via <code>renderOptions</code>	One focus line per beat
<code>seam</code>	whole-line top border, 2px dashed	Head line just after a removal (fallback only)
<code>companionRemoved</code>	whole-line background from <code>diffEditor.removedLineBackground</code>	Removed lines in the companion editor

Focus lines also carry a `hoverMessage` (a trusted `MarkdownString`) with the beat narration and `command:` links to open the claim drawer.

Label placement

`after` content renders at the end of the line's text, so a long line can push the label off screen. The label goes on the shortest of the first three lines of the first focus span. The panel always shows the same text, so a clipped label loses nothing.

Theme colors

```
"contributes": {
  "colors": [
    { "id": "relay.presenterRail", "description": "Tour presenter rail and
overview ruler",
      "defaults": { "dark": "#A99BF5", "light": "#534AB7", "highContrast":
"#C9BFFF", "highContrastLight": "#26215C" } },
    { "id": "relay.presenterFocus", "description": "Tour focus outline",
      "defaults": { "dark": "#A99BF5", "light": "#534AB7", "highContrast":
"#FFFFFF", "highContrastLight": "#000000" } },
    { "id": "relay.presenterLabelBackground", "description": "Beat label
background",
      "defaults": { "dark": "#3C3489", "light": "#EEEDFE", "highContrast":
"#000000", "highContrastLight": "#FFFFFF" } },
    { "id": "relay.presenterLabelForeground", "description": "Beat label
text",
      "defaults": { "dark": "#EEEDFE", "light": "#3C3489", "highContrast":
"#FFFFFF", "highContrastLight": "#000000" } },
    { "id": "relay.presenterSeam", "description": "Marker where removed lines
were",
      "defaults": { "dark": "#A99BF5", "light": "#534AB7", "highContrast":
"#FFFFFF", "highContrastLight": "#000000" } }
  ]
}
```

`companionRemoved` reuses the workbench color `diffEditor.removedLineBackground` instead of contributing its own, so the companion editor matches the user's diff theme exactly.

Reference implementation

```
import * as vscode from "vscode";

const tc = (id: string) => new vscode.ThemeColor(id);
const line = (l: number) => new vscode.Range(1, 0, 1, 0);
const fullLine = (l: number) => new vscode.Range(1, 0, 1,
Number.MAX_SAFE_INTEGER);

function box(width: string) {
  return vscode.window.createTextEditorDecorationType({
    isWholeLine: true,
```

```

    borderStyle: "solid",
    borderColor: tc("relay.presenterFocus"),
    borderWidth: width,
  });
}

export const types = {
  rail: vscode.window.createTextEditorDecorationType({
    isWholeLine: true,
    borderStyle: "solid",
    borderColor: tc("relay.presenterRail"),
    borderWidth: "0 0 0 3px",
    overviewRulerColor: tc("relay.presenterRail"),
    overviewRulerLane: vscode.OverviewRulerLane.Left,
  }),
  dim: vscode.window.createTextEditorDecorationType({ opacity: "0.45" }),
  boxTop: box("1px 1px 0 1px"),
  boxMid: box("0 1px 0 1px"),
  boxBot: box("0 1px 1px 1px"),
  boxOne: box("1px"),
  label: vscode.window.createTextEditorDecorationType({
    after: {
      margin: "0 0 0 2em",
      color: tc("relay.presenterLabelForeground"),
      backgroundColor: tc("relay.presenterLabelBackground"),
    },
  }),
  seam: vscode.window.createTextEditorDecorationType({
    isWholeLine: true,
    borderStyle: "dashed",
    borderColor: tc("relay.presenterSeam"),
    borderWidth: "2px 0 0 0",
  }),
  companionRemoved: vscode.window.createTextEditorDecorationType({
    isWholeLine: true,
    backgroundColor: tc("diffEditor.removedLineBackground"),
  }),
};

export interface Paint {
  context: vscode.Range;
  focus: vscode.Range[]; // already resolved to this editor's side
  label?: string;
}
```

```

    hover?: vscode.MarkdownString;
    dimOutsideFocus: boolean;
    seamLines?: number[];
    removedLines?: number[];          // companion editor only
}

export function paint(editor: vscode.TextEditor, p: Paint) {
    clear(editor);
    const ctxLines = lines(p.context);
    editor.setDecorations(types.rail, ctxLines.map(line));

    const top: vscode.DecorationOptions[] = [];
    const mid: vscode.DecorationOptions[] = [];
    const bot: vscode.DecorationOptions[] = [];
    const one: vscode.DecorationOptions[] = [];
    for (const f of p.focus) {
        const deco = (l: number) => ({ range: line(l), hoverMessage: p.hover });
        if (f.start.line === f.end.line) { one.push(deco(f.start.line));
continue; }
        top.push(deco(f.start.line));
        bot.push(deco(f.end.line));
        for (let l = f.start.line + 1; l < f.end.line; l++) mid.push(deco(l));
    }
    editor.setDecorations(types.boxTop, top);
    editor.setDecorations(types.boxMid, mid);
    editor.setDecorations(types.boxBot, bot);
    editor.setDecorations(types.boxOne, one);

    if (p.dimOutsideFocus && p.focus.length) {
        const inFocus = (l: number) => p.focus.some(f => l >= f.start.line && l
<= f.end.line);
        editor.setDecorations(types.dim, ctxLines.filter(l =>
!inFocus(l)).map(fullLine));
    }

    if (p.label && p.focus.length) {
        editor.setDecorations(types.label, [{
            range: line(labelLine(editor.document, p.focus[0])),
            renderOptions: { after: { contentText: `\\u2002${p.label}\\u2002` } },
        }]);
    }

    if (p.seamLines) editor.setDecorations(types.seam, p.seamLines.map(line));
}

```

```

    if (p.removedLines) editor.setDecorations(types.companionRemoved,
p.removedLines.map(line));
  }

export function clear(editor: vscode.TextEditor) {
  for (const t of Object.values(types)) editor.setDecorations(t, []);
}

function lines(r: vscode.Range): number[] {
  const out: number[] = [];
  for (let l = r.start.line; l <= r.end.line; l++) out.push(l);
  return out;
}

function labelLine(doc: vscode.TextDocument, f: vscode.Range): number {
  const candidates = lines(f).slice(0, 3);
  return candidates.reduce((best, l) =>
    doc.lineAt(l).text.length < doc.lineAt(best).text.length ? l : best,
candidates[0]);
}

```

Presentation states

The decorations depend on the follow state from the panel spec. The rule: the more the reviewer is steering, the less the tour paints.

State	Rail	Focus box	Dim	Label	Seam / companion
Following	Yes	Yes	Yes	Yes	Yes
Exploring	Yes	Yes	No	No	Companion stays open, not re-revealed
Detour, answer pointing at code	On the detour anchor	Yes	No	Detour title	As needed
Paused	Yes	Yes	No	Yes	Unchanged
Stale beat	Dashed, via a separate railStale type (border style is fixed per type)	No	No	"Stale"	No

Following. Rail, focus box, dim, label.

scheduler.go (a41c9e) ↔ scheduler.go (7f02b1)

```

153     managed := make([]db.Node, 0, len(pendingNodes))
154     fmt.Println("Test adding a line to see how it shows up in the diff")
155     for _, node := range pendingNodes {
156         if !node.IsManagedByControlPlaneController() {
157             continue
158         } Beat 2 · C3 inferred
159         managed = append(managed, node)
160     }
161     return managed, nil
162 }

```

Exploring. Reviewer scrolled or selected elsewhere. Rail and box stay as a way back; no dim, no label, editor never moved.

scheduler.go (a41c9e) ↔ scheduler.go (7f02b1)

```

153     managed := make([]db.Node, 0, len(pendingNodes))
154     fmt.Println("Test adding a line to see how it shows up in the diff")
155     for _, node := range pendingNodes {
156         if !node.IsManagedByControlPlaneController() {
157             continue
158         }
159         managed = append(managed, node)
160     }
161     return managed, nil
162 }

```

Detour. Agent answer points at the added line. Detour anchor replaces the beat decorations until Return.

scheduler.go (a41c9e) ↔ scheduler.go (7f02b1)

```

153     managed := make([]db.Node, 0, len(pendingNodes))
154     fmt.Println("Test adding a line to see how it shows up in the diff") Detour · Why print here?
155     for _, node := range pendingNodes {
156         if !node.IsManagedByControlPlaneController() {
157             continue
158         }
159         managed = append(managed, node)
160     }
161     return managed, nil
162 }

```

Stale. Anchor hash no longer matches. Dashed rail, stale label, no focus box or dim.

scheduler.go (a41c9e) ↔ scheduler.go (7f02b1)

```

153     managed := make([]db.Node, 0, len(pendingNodes))
154     fmt.Println("Test adding a line to see how it shows up in the diff")
155     for _, node := range pendingNodes {
156         if !node.IsManagedByControlPlaneController() {
157             continue
158         } Stale · code changed since beat
159         managed = append(managed, node)
160     }
161     return managed, nil
162 }

```

Rules

- Dimming only happens while Following. Fading the code a reviewer chose to look at works against them.
- The rail never disappears while a tour is loaded. It's how a reviewer who wandered off finds their way back, together with the overview ruler mark.
- Detour anchors replace the beat decorations while the detour is open, then the beat decorations are repainted on Return. Nothing from a detour lingers.

- Painting is idempotent per editor. On any state change the extension calls `clear` then `paint` on each affected visible editor instead of diffing decoration sets.

When to repaint

Event	Action
Beat activated	Paint the target editor(s), clear all others
<code>onDidChangeVisibleTextEditors</code>	Repaint any newly visible editor showing a tour URI; decorations don't survive an editor being closed and reopened
Follow state change	Repaint the active beat's editors
<code>onDidChangeTextDocument</code> on a worktree anchor	Re-hash on a 300 ms debounce; repaint or mark stale
<code>onDidChangeConfiguration</code> for <code>diffEditor.*</code> or <code>relay.presentation.*</code>	Re-run the removed-code strategy for the current beat

Removed-code focus

A beat with a `base`-side focus span needs a real editor showing base text. Relay opens the diff, checks whether VS Code rendered it side by side, and decorates the original-side editor if so. If the diff rendered inline, Relay opens a companion editor with the base file beside it, or, if the user prefers, marks the removal point on the head side with a seam.

flowchart TD

```

A[Beat activated] --> B{Any base-side<br/>focus span?}
B -- no --> H[Paint head editor only]
B -- yes --> C[vscode.diff base, head]
C --> D{Original editor in<br/>visibleTextEditors?}
D -- yes --> S[Side by side:<br/>paint both sides]
D -- no --> E{relay.presentation<br/>.removedCode}
E -- companion --> F[Seam on head +<br/>base editor Beside]
E -- seam --> G[Seam on head +<br/>peek link in hover]
```

The diff opens in every case. The companion supplements the inline diff and never replaces it.

Detecting side by side

There is no API to ask a diff editor for its layout, and no per-diff option to force one.

`diffEditor.renderSideBySide` is a user setting, and `diffEditor.useInlineViewWhenSpaceIsLimited` (on by default) flips a side-by-side diff to inline when the editor group is narrow. That's likely with the Relay panel in the secondary sidebar, so reading settings can't predict the layout.

The runtime state is observable, though. A side-by-side diff exposes both sides in `vscode.window.visibleTextEditors`, and an inline diff exposes only the modified side. After `vscode.diff` resolves, Relay waits up to 500 ms for an editor whose `document.uri` matches the base `relay-rev:` URI.

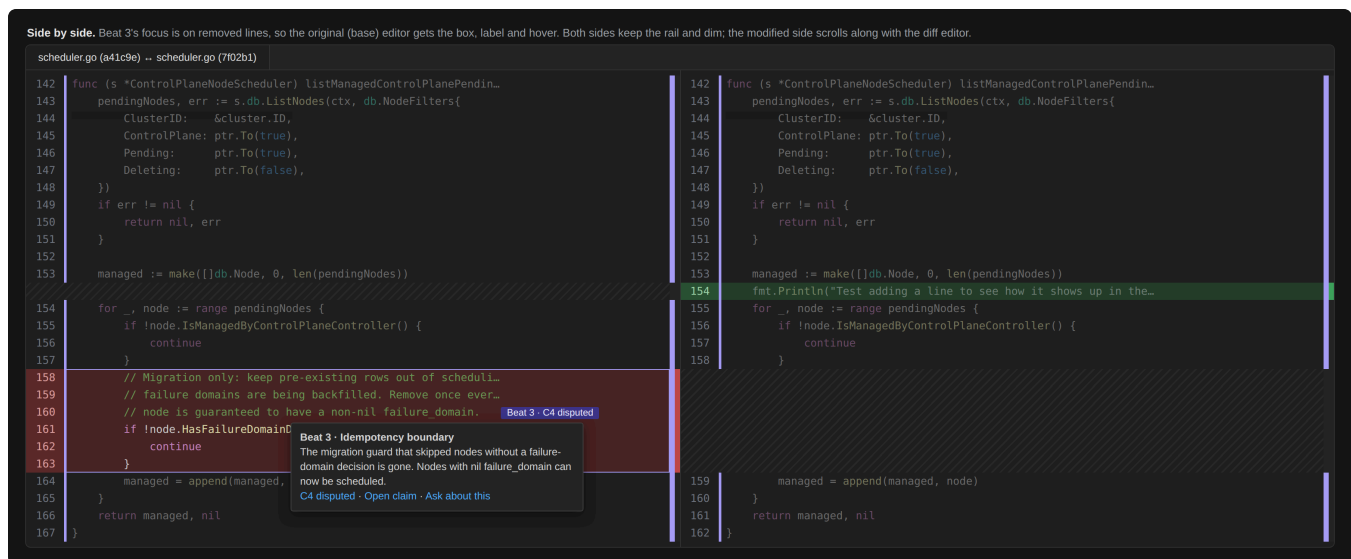
```
async function openDiff(baseUri: vscode.Uri, headUri: vscode.Uri, title:
string) {
    await vscode.commands.executeCommand("vscode.diff", baseUri, headUri,
title, {
        preview: true,
        preserveFocus: false,
    });
    const find = () => ({
        base: vscode.window.visibleTextEditors.find(e =>
e.document.uri.toString() === baseUri.toString()),
        head: vscode.window.visibleTextEditors.find(e =>
e.document.uri.toString() === headUri.toString()),
    });
    let eds = find();
    if (eds.base && eds.head) return { ...eds, layout: "sideBySide" as const };

    await new Promise<void>(resolve => {
        const timer = setTimeout(() => { sub.dispose(); resolve(); }, 500);
        const sub = vscode.window.onDidChangeVisibleTextEditors(() => {
            eds = find();
            if (eds.base && eds.head) { clearTimeout(timer); sub.dispose();
resolve(); }
        });
    });
    eds = find();
    return { ...eds, layout: eds.base ? "sideBySide" as const : "inline" as
const };
}
```

Layout can change without a beat change: the user resizes the window, toggles the setting, or closes the sidebar. `onDidChangeVisibleTextEditors` and `onDidChangeConfiguration` (for `diffEditor.*`) re-run detection for the current beat and repaint.

Side by side

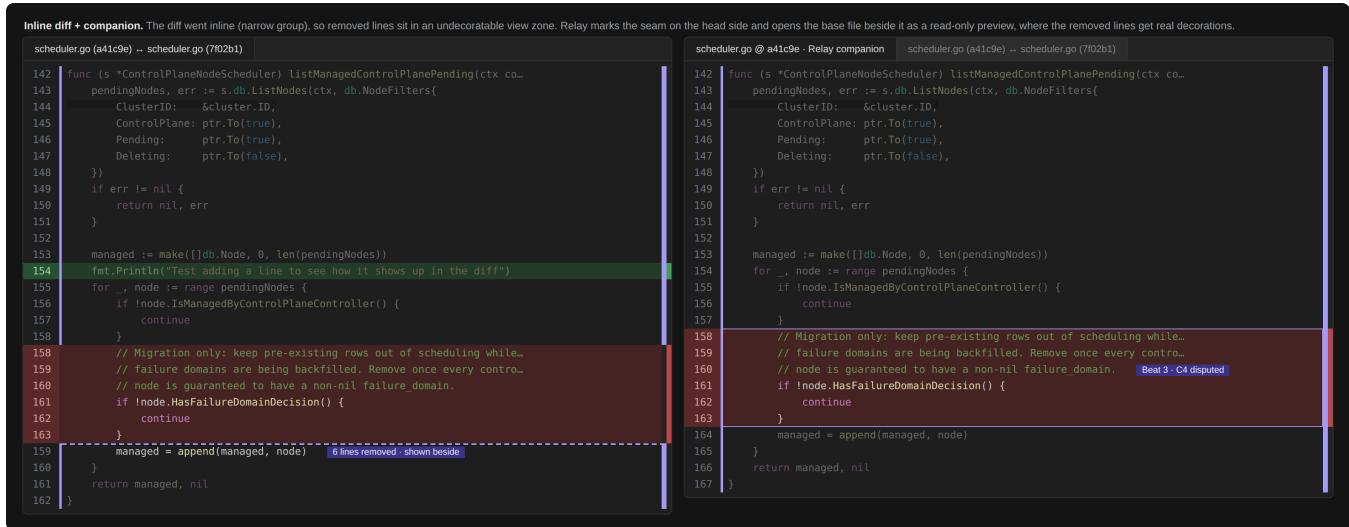
- Paint the original editor with base-side focus spans (box, label, hover) and the rail over the context range mapped to base lines.
- Paint the modified editor with head-side focus spans and the rail over the head context range.
- Dim applies on both sides.
- `revealRange` on the side holding the first focus span with `TextEditorRevealType.InCenterIfOutsideViewport`. The diff editor's synchronized scrolling brings the other side along.



Inline: companion editor (default)

1. Paint the head editor as usual, plus a `seam` on the head line where the removed lines used to be, with a label such as "6 lines removed · shown beside".
2. Open the base file with `showTextDocument(baseUri, { viewColumn: ViewColumn.Beside, preserveFocus: true, preview: true })`. The URI carries `?relay=companion` so Relay can recognize its own tabs.
3. In the companion, paint `companionRemoved` backgrounds on every removed line from the file's hunks, then the base-side focus box, label, and rail, and reveal the focus.

- The companion is reused across beats in the same file. When the tour moves to a beat with no base-side focus, or to another file, Relay closes it with `vscode.window.tabGroups.close(tab)`. It only ever closes tabs whose URI carries the `relay=companion` marker.
- If the reviewer pins the companion (turns off preview mode) or edits its layout, Relay stops managing it and leaves it open.



The companion is read-only (a `TextDocumentContentProvider` document), so there's no risk of the reviewer editing base text by accident.

Inline: seam only

For reviewers who don't want a split, `relay.presentation.removedCode: "seam"` skips the companion. The seam's hover carries a command link that runs the documented built-in `editor.action.showReferences` with the base URI and focus location. That opens a peek widget with the removed code inline in the diff editor. The peek can't be decorated, but it shows the exact lines with syntax highlighting and closes with Escape.

Mapping removed lines to a seam

Relay already has base and head shas, so it runs `git diff -U0 <base> <head> -- <path>` once per file per tour and caches the parsed hunks. With zero context lines, a pure deletion hunk reads `@@ -a, b +c, 0 @@`, meaning b base lines starting at line a were removed after head line c. The seam goes on head line c + 1 (1-based).

```

interface Hunk { baseStart: number; baseLen: number; headStart: number;
headLen: number }

const HUNK = /^@@ -(\d+)(?:,(\d+))? \+(\d+)(?:,(\d+))? @@/;

export function parseHunks(diffText: string): Hunk[] {
  const out: Hunk[] = [];
  for (const l of diffText.split("\n")) {
    const m = HUNK.exec(l);
    if (!m) continue;
    out.push({
      baseStart: +m[1], baseLen: m[2] === undefined ? 1 : +m[2],
      headStart: +m[3], headLen: m[4] === undefined ? 1 : +m[4],
    });
  }
  return out;
}

// 0-based head line where a seam for a removed base span should sit.
export function seamLineFor(baseSpanStart0: number, hunks: Hunk[]): number |
undefined {
  const baseLine1 = baseSpanStart0 + 1;
  const h = hunks.find(h => baseLine1 >= h.baseStart && baseLine1 <
h.baseStart + h.baseLen);
  if (!h) return undefined;
  // Pure deletion: seam on the line after headStart. Replacement: seam on
the first new line.
  return h.headLen === 0 ? h.headStart : h.headStart - 1;
}

export function removedBaseLines0(hunks: Hunk[]): number[] {
  return hunks.flatMap(h => Array.from({ length: h.baseLen }, (_, i) =>
h.baseStart - 1 + i));
}

```

When the removed span sits at the end of the file, `seamLineFor` can return a line equal to the document's line count. Clamp to the last line and draw the seam as a bottom border instead.

VS Code API constraints

Everything in this spec uses stable, public extension API plus built-in commands (`vscode.diff`, `editor.action.showReferences`). No proposed APIs, no private DOM access, and no custom editors.

Constraint	Consequence for this design
Extensions can't decorate view zones, and inline-diff removed lines live in view zones	Removed code needs the original-side editor or a companion editor
No API to read or force a diff editor's layout per instance	Detect layout from <code>visibleTextEditors</code> ; never assume from settings
<code>diffEditor.useInlineViewWhenSpaceIsLimited</code> switches to inline in narrow groups	Detection runs after every visible-editor change, not just at open
Extensions can't insert phantom lines (view zones aren't exposed)	The "lines removed" marker is a border plus an <code>after</code> label on a real line
<code>renderOptions</code> per range only covers <code>before / after</code> , not borders or opacity	Each border style or opacity level is its own decoration type
CodeLens is off in diff editors by default (<code>diffEditor.codeLens</code>)	Beat labels use <code>after</code> attachments, which render everywhere
Decorations are per editor instance and vanish when an editor closes	Repaint on <code>onDidChangeVisibleTextEditors</code>
Peek widgets opened via <code>showReferences</code> aren't exposed as <code>TextEditor</code> s	The seam-only mode's peek shows removed code undecorated

Rejected approaches

- **Flipping `diffEditor.renderSideBySide` automatically.** It writes the user's settings file and still loses to narrow-width auto-inline. The panel can offer a one-click "Show diff side by side" button that runs the built-in `toggle.diff.renderSideBySide` command, but only as an explicit reviewer action.
- **A custom diff webview.** Full control over rendering, but it throws away language features, go-to-definition, the user's theme and keybindings, and every other extension's decorations. Too high a price for reviewer trust in "this is my editor."

- **Gutter icons for the rail.** `gutterIconPath` competes with breakpoints, Git change bars, and test status icons in a narrow column, and can't span lines continuously. The left content border is always free.
- **Background tint at low alpha.** Any background, however faint, blends with diff colors and shifts them. The screenshot that motivated this spec is the evidence.

Settings and open questions

Settings

Setting	Type	Default	Effect
<code>relay.presentation.removedCode</code>	"companion" "seam"	"companion"	How base-side focus is shown when the diff renders inline
<code>relay.presentation.dimOpacity</code>	number, 0.2-1.0	0.45	Opacity of context text outside the focus; 1.0 disables dimming
<code>relay.presentation.showLabels</code>	boolean	true	Beat labels on focus lines
<code>relay.presentation.closeCompanion</code>	"onBeatChange" "onTourEnd" "never"	"onBeatChange"	When Relay closes companion tabs it opened

Dim opacity is fixed per decoration type, so changing it disposes and recreates `dim`, then repaints.

Open questions

- ☐ **Companion placement.** `ViewColumn.Beside` splits an editor area that may already be narrow, which is often why the diff went inline in the first place. An alternative is opening the companion in a new group below (`workbench.action.newGroupBelow` , then show in it). Worth testing both at common laptop widths with the sidebar open.
- ☐ **Scroll sync for the companion.** The companion doesn't scroll with the diff. Relay could mirror `onDidChangeTextEditorVisibleRanges` from head to base through the hunk map. It's doable, but may fight the reviewer if they scroll the companion on purpose. Start without it.

- ☐ **Focus spans on both sides in one beat.** Side by side handles this well. Inline plus companion shows both, but in two places. Should the label call out "also beside"?
- ☐ **Stale focus, fresh context.** If the context hash matches but a focus span's text moved within it, should Relay re-anchor the focus by content search inside the context before falling back to stale?
- ☐ **Theme contrast.** Violet `#A99BF5` on dark and `#534AB7` on light need a check against popular themes (One Dark, GitHub Light, Solarized), especially where themes tint the whole editor background.